



Learning Objectives



Mahasiswa mampu:

01



Memahami konsep **Unit Testing**

02



Menjelaskan pentingnya **otomatisasi testing**

03

JUnit

Menggunakan framework **JUnit**

04



Menulis dan menjalankan **test case otomatis**



PENGERTIAN UNIT TESTING

Unit Testing adalah:

Pengujian terhadap bagian terkecil dari program
(fungsi/method/class)

Fokus:



Logika
program



Fungsi
individual

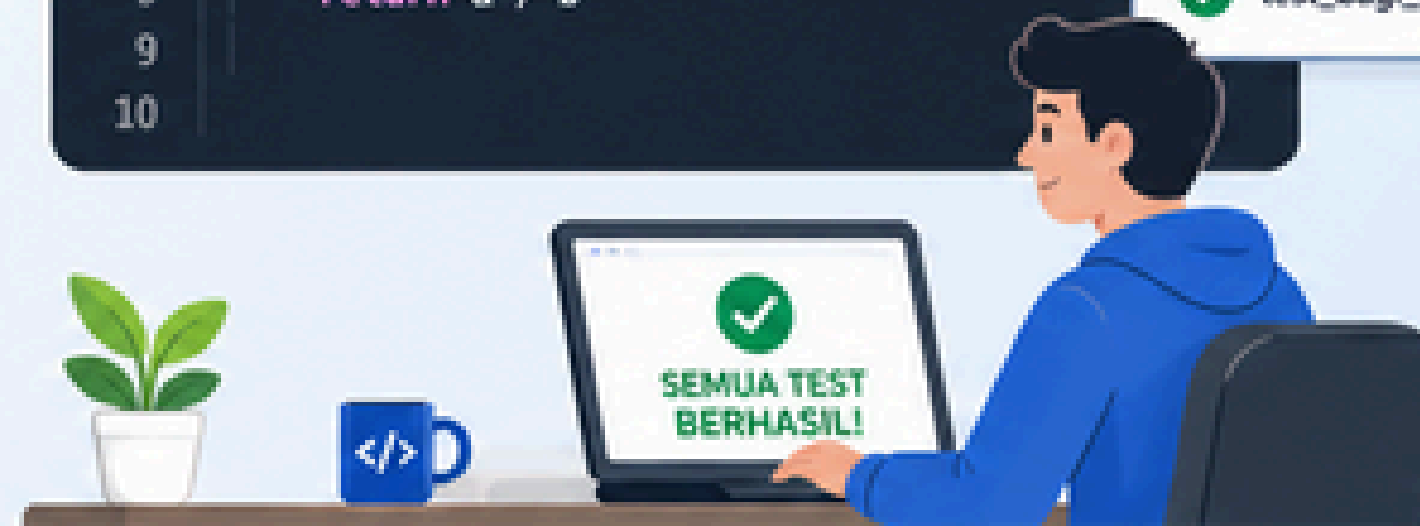


Validasi
input/output

```
calculator.py
1 def tambah(a, b):
2     return a + b
3
4 def bagi(a, b):
5     if b == 0:
6         raise ValueError("b tidak boleh 0")
7     return a / b
8
9
10
```

TEST HASIL

✓ test_tambah_posif	PASS
✓ test_tambah_negatif	PASS
✓ test_bagi_normal	PASS
✓ test_bagi_pembagi_nol	PASS



KARAKTERISTIK UNIT TESTING



Dilakukan oleh developer



Skala kecil (fungsi/method)



Cepat dijalankan



Biasanya otomatis



TUJUAN UNIT TESTING



Menemukan bug sejak awal



Memastikan fungsi berjalan benar



Mendukung refactoring



Dokumentasi perilaku kode

KENAPA UNIT TESTING PENTING?

- ✓ Mengurangi bug di production
- ✓ Meningkatkan kualitas kode
- ✓ Mempermudah maintenance
- ✓ Membangun kepercayaan saat pengembangan



"Test kecil hari ini, mencegah **bug** besar esok hari."

PERBEDAAN MANUAL vs AUTOMATED TESTING

(DALAM UNIT TESTING)



MANUAL TESTING

Pengujian dilakukan secara manual oleh manusia tanpa bantuan tool otomatis.



AUTOMATED TESTING

Pengujian dilakukan menggunakan script/program (misalnya JUnit) yang berjalan otomatis.



ASPEK	MANUAL TESTING	AUTOMATED TESTING
 CARA KERJA	 Dijalankan manual oleh tester	 Dijalankan oleh program
 KECEPATAN	 Lambat	 Sangat cepat
 KONSISTENSI	 Bisa berubah (human error)	 Konsisten
 REUSABILITY	 Sulit diulang	 Mudah diulang
 SKALA	 Terbatas	 Bisa besar & massal
 COCOK UNTUK	 Exploratory testing	 Unit testing



Kesimpulan: Manual testing penting untuk eksplorasi, sedangkan automated testing penting untuk kecepatan, konsistensi, dan skalabilitas dalam unit testing.





PERBANDINGAN MANUAL vs AUTOMATED TESTING

DALAM UNIT TESTING



MANUAL TESTING (Dilakukan oleh manusia)

VS



AUTOMATED TESTING (Dilakukan oleh program / tools)



Kecepatan

Lambat



KECEPATAN



Sangat cepat



Akurasi & Konsistensi

Rentan human error,
hasil bisa tidak konsisten



AKURASI &
KONSISTENSI



Konsisten dan akurat,
tidak terpengaruh human error

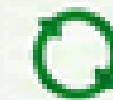


Reusability
(Dapat Digunakan Ulang)

Sulit diulang



REUSABILITY



Mudah diulang kapan saja

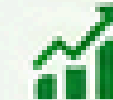


Skalabilitas

Terbatas, sulit
untuk test dalam jumlah besar



SKALABILITAS



Bisa test skala besar dengan mudah



Biaya & Waktu

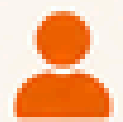
Lebih mahal dalam jangka panjang



BIAYA & WAKTU



Lebih hemat dalam jangka panjang

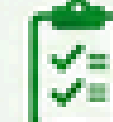


Cocok Untuk

- Exploratory testing
- Testing UI/UX
- Skenario yang berubah-ubah / tidak pasti
- Validasi visual (layout, warna, responsif)



COCOK UNTUK



- Unit Testing (WAJIB / sangat direkomendasikan)
- Regression testing
- Testing yang sering dijalankan
- Continuous Integration / CI-CD
- Pengujian skala besar



KAPAN DIGUNAKAN?

Gunakan Manual Testing ketika:

- Exploratory testing (menjelajahi aplikasi)
- Testing UI/UX (tampilan & pengalaman pengguna)
- Skenario yang berubah-ubah / tidak pasti
- Validasi visual (layout, warna, responsif)

RULE OF THUMB



Jika test sering diulang → gunakan Automated



Jika butuh intuisi manusia → gunakan **Manual**



Jika unit testing → **HARUS Automated**



Jika UI/UX testing → **Manual lebih cocok**

Gunakan Automated Testing ketika:

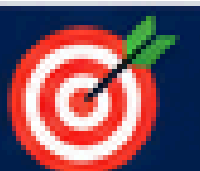
- Unit Testing (WAJIB / sangat direkomendasikan)
- Regression testing (pengujian berulang)
- Testing yang sering dijalankan
- Continuous Integration / CI-CD
- Pengujian skala besar



KESIMPULAN:

Manual testing = fleksibel & eksploratif | Automated testing = cepat, konsisten & berulang

Dalam **UNIT TESTING** → **AUTOMATED TESTING** adalah standar utama!



Apa itu JUnit?

JUnit

JUnit adalah **framework pengujian** untuk bahasa Java yang digunakan untuk menulis, menjalankan, dan mengotomatisasi unit testing.

JUnit membantu **developer** memastikan bahwa method, class, atau komponen kecil program berjalan sesuai harapan.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {
    @Test
    void tambah_dua_angka() {
        Calculator calc = new Calculator();
        int hasil = calc.tambah(2, 3);
        assertEquals(5, hasil);
    }
}
```



Menurut dokumentasi resmi, JUnit adalah **framework testing** untuk Java/JVM yang mendukung berbagai gaya pengujian dan terintegrasi dengan **IDE** serta build tools seperti **Maven** dan **Gradle**.



JUnit 5



Maven



Fungsi Utama JUnit

1



Membuat test case otomatis

Developer dapat menulis kode pengujian yang bisa dijalankan berulang kali.

2



Memeriksa hasil program dengan assertion

Contoh: membandingkan hasil aktual dengan hasil yang diharapkan menggunakan `assertEquals()`.

3



Mendeteksi bug lebih awal

Kesalahan pada fungsi kecil dapat ditemukan sebelum sistem menjadi besar.

4



Mendukung regression testing

Test dapat dijalankan ulang setelah perubahan kode untuk memastikan fitur lama tetap berjalan.

5



Mendukung Test-Driven Development (TDD)

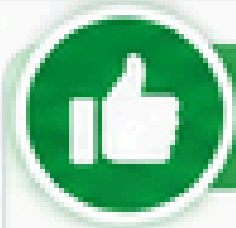
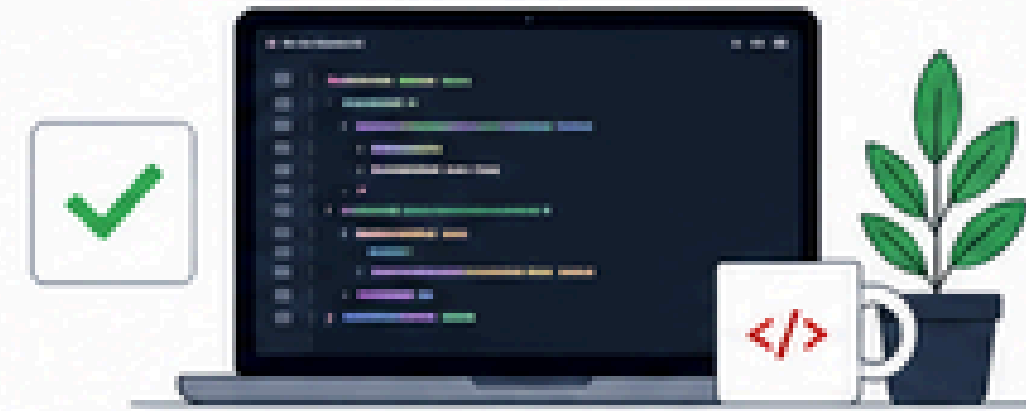
JUnit sangat erat dengan praktik TDD, yaitu menulis test terlebih dahulu sebelum implementasi kode.



Singkatnya, JUnit membantu developer menulis kode Java yang lebih berkualitas, andal, dan mudah dipelihara.

JUnit

Framework testing unit untuk **Java** yang sederhana, powerful, dan populer.



KELEBIHAN JUNIT



Open source

JUnit bersifat open source dan gratis digunakan oleh siapa saja.



Mudah digunakan

API sederhana dan annotation yang mudah dipahami.



Terintegrasi dengan IDE

Terintegrasi dengan IDE populer seperti IntelliJ IDEA dan Eclipse.



Mendukung otomatisasi

Mudah dijalankan otomatis melalui build tool (Maven, Gradle) dan CI/CD pipeline.



KEKURANGAN JUNIT



Hanya untuk Java

JUnit dirancang khusus untuk bahasa pemrograman Java.



Butuh pemahaman coding

Pengguna perlu memahami konsep pemrograman dan testing.



Tidak cocok untuk UI testing

JUnit bukan tools untuk UI testing. Gunakan tools lain seperti Selenium.



BEST PRACTICES



Satu test untuk satu skenario

Setiap test harus fokus pada satu skenario atau satu perilaku tertentu.

```
testNameValid()
```

Gunakan nama test yang jelas

Nama test harus deskriptif sehingga mudah dipahami tanpa harus membaca isi test.



Hindari dependency antar test

Test harus independen. Hasil satu test tidak boleh mempengaruhi test lainnya.



Gunakan setup & cleanup

Gunakan `@BeforeEach` / `@BeforeAll` untuk setup dan `@AfterEach` / `@AfterAll` untuk cleanup.



JUnit membantu memastikan kode Anda bekerja dengan benar, memudahkan maintenance, dan meningkatkan kualitas software.



Contoh Implementasi JUnit Lebih Kompleks

Mocking • Parameterized Test • Assertions • Test Lifecycle



JUnit 5



Mockito



Java 17+

UserServiceTest.java

```
1  @ExtendWith(MockitoExtension.class)
2  @TestInstance(TestInstance.Lifecycle.PER_CLASS)
3  class UserServiceTest {
4
5      @Mock
6      private UserRepository userRepository;
7
8      @InjectMocks
9      private UserService userService;
10     private AutoCloseable mocks;
11 }
12
13 @BeforeAll
14 void setUpAll() {
15     mocks = MockitoAnnotations.openMocks(this);
16 }
17
18 @AfterAll
19 void tearDownAll() throws Exception {
20     mocks.close();
21 }
22
23 @ParameterizedTest(name = "{index} => email: {0}, expected: {1}")
24 @CsvSource({
25     "user@example.com, true",
26     "USER@EXAMPLE.COM, true",
27     "invalid-email, false",
28     "", false"
29 })
30 void shouldReturnTrueForValidEmail(String email, boolean expected) {
31     // Stubbing behavior
32     when(userRepository.existsByEmail(email.toLowerCase())).thenReturn(expected);
33
34     boolean result = userService.isEmailAvailable(email);
35     assertEquals(expected, result);
36     verify(userRepository, times(1)).existsByEmail(email.toLowerCase());
37     verifyNoMoreInteractions(userRepository);
38 }
39
40 @Test
41 void shouldThrowExceptionWhenUserNotFound() {
42     when(userRepository.findById(1L)).thenReturn(Optional.empty());
43
44     NotFoundException ex = assertThrows(NotFoundException.class,
45         () -> userService.getUserById(1L));
46
47     assertTrue(ex.getMessage().contains("User not found"));
48     verify(userRepository).findById(1L);
49 }
```

1. MOCKING (Mockito)

Membuat mock dependency dan inject ke service yang akan diuji.

2. TEST LIFECYCLE

Inisialisasi dan cleanup resource untuk seluruh test di class ini.

3. PARAMETERIZED TEST

Menjalankan test dengan berbagai input (data-driven). Cocok untuk skenario yang memiliki banyak variasi data.

4. ASSERTIONS & VERIFY

Berbagai assertion untuk validasi hasil, serta verify interaksi mock. assertThrows untuk exception test.

SYSTEM UNDER TEST



FITUR YANG DIGUNAKAN



Mocking (Mockito)

Mengisolasi unit yang diuji dengan memalsukan dependency.



Parameterized Test

Menjalankan test berulang dengan berbagai parameter input.



Assertions

Memvalidasi output, state, atau exception yang diharapkan.



Test Lifecycle

Mengelola setup dan teardown resources secara efisien.

STRUKTUR PROYEK (contoh)



```
Dependency (Maven)
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.2</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-junit-jupiter</artifactId>
    <version>5.11.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

\$ mvn test

BUILD SUCCESS

Test run: 5, Failures: 0, Errors: 0, Skipped: 0



Contoh ini menunjukkan kombinasi fitur JUnit 5 dan Mockito untuk unit test yang lebih realistis dan maintainable.

JUnit

adalah framework standar untuk unit testing di Java.

Dengan JUnit, developer dapat membuat test otomatis, memvalidasi hasil program, menemukan bug lebih cepat, dan menjaga kualitas kode saat aplikasi berkembang.



Test Otomatis

Membuat test yang berjalan otomatis tanpa perlu pengecekan manual.



Validasi Hasil Program

Memastikan output sesuai harapan dan berperilaku benar.



Temukan Bug Lebih Cepat

Test yang terstruktur membantu menemukan kesalahan sejak dini.



Jaga Kualitas Kode

Memastikan kode tetap stabil dan mudah dikembangkan seiring waktu.



JUnit membantu developer menulis kode dengan percaya diri, karena setiap perubahan teruji secara otomatis.

```
CalculatorTest.java
1 import static org.junit.jupiter.api.Assertions.*;
2 import org.junit.jupiter.api.Test;
3
4 class CalculatorTest {
5     private final Calculator calc = new Calculator();
6
7     @Test
8     void tambah_dua_angka() {
9         int hasil = calc.tambah(2, 3);
10        assertEquals(5, hasil);
11    }
12 }
```



Tests passed: 1 / 1

Time: 0.02 s





Thank You!

See You Next Week